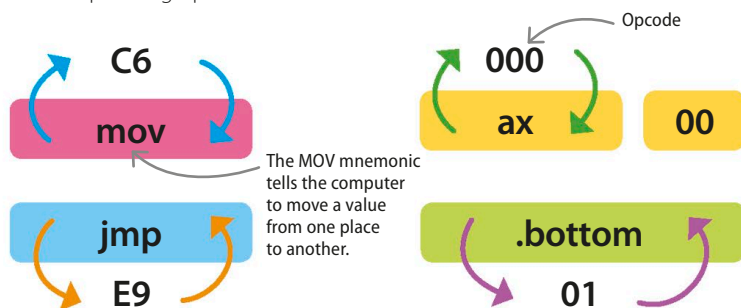


Assemblers, interpreters, and compilers

Translators turn high-level code into machine code, and there are three main types: assemblers, interpreters, and compilers.

Assemblers

The only difference between assembly code and opcode – the instructions to the computer to perform an operation – are the names of instructions. Assembly code is a low-level language that is written in mnemonics. Mnemonics are simple instructions that are easier for humans to remember than opcode. An assembler goes through code written in assembly code and simply swaps every mnemonic it finds for its corresponding opcode.



△ Assembling

An assembler is the most basic type of compiler. It simply swaps out mnemonics with the specific set of opcodes belonging to its operating system.

SEE ALSO

◀ 44–45 Operating systems

◀ 110–111 Translation

JavaScript

134–135 ▶

IN DEPTH

Assembly

Assembly is commonly used in applications such as mobile phone chips, ATMs, and video-game consoles, where space and speed are important.

Pros

- Programmers don't have to remember opcodes.
- A single program can work on multiple computers.

Cons

- Programmers still work at a low level with registers, stack pointers, and heaps.
- An assembler has to be written for each OS.
- Even simple programs have many lines of code.

Interpreters

Interpretive languages can't be run without their interpreter installed on the computer. An interpreter translates and executes the source code one line at a time. While this makes code more portable, which means it can be easily adapted to run on a different operating system, it also makes it slower. In general, it's harder to guarantee that a program is error-free, but it's easier to fix errors when they crop up.

▷ Easy to handle

The interpretive style makes it easy to handle interactive online pages as it can receive new code any time and still run it.

